Comparing Ways to Speed Up the Convolution Layer of a CNN on the GPU

Final Report

Parallel and Distributed Computing, Group Project

Group Member	Roll Number
Faizan Tanveer	23L-0640
Mania Ali	23L-0876
Khadija Fatima	23L-0976

November 2026

Abstract

Convolution dominates the running time of convolutional neural networks, and several ways of arranging it for a GPU exist, but they are rarely compared fairly on one machine. We implemented im2col with a hand-written matrix multiplication, implicit GEMM and tiled direct convolution in CUDA, and compared them with cuBLAS, cuDNN and CPU baselines on an RTX 4050 Laptop GPU. We measured time and memory across 23 layers, explained the results with a roofline model, and timed end-to-end runs against OpenMP. All seven versions passed all 29 correctness configurations. cuDNN was fastest on 20 of 23 layers. Among our own methods, implicit GEMM was fastest on 18 of 23 layers and used the least memory, needing 196 MB where im2col needed 1,080 MB on the largest layer. Direct convolution matched that memory use but was usually slower. im2col methods were mostly memory-bound, while implicit GEMM and direct convolution were limited by their own overheads rather than by memory bandwidth. Every GPU method beat OpenMP end to end on all synthetic layers. We recommend implicit GEMM for most layers and im2col only for small, many-channel inputs.

Contents

1 Introduction **3**
- 1.1 Background . . . 3
- 1.2 Problem Statement . . . 3
- 1.3 Research Gaps . . . 3
- 1.4 Research Question and Objectives . . . 4
- 1.5 Scope . . . 4

2 Literature Review **4**
- 2.1 Convolution as Matrix Multiplication . . . 4
- 2.2 Implicit GEMM . . . 4
- 2.3 Memory-Efficient Convolution . . . 5
- 2.4 Transform Methods . . . 5
- 2.5 Performance Modelling . . . 5
- 2.6 Summary . . . 5

3 Methodology **5**
- 3.1 Research Design . . . 5
- 3.2 Implementations . . . 6
- 3.3 Experimental Setup . . . 6
- 3.4 Workloads . . . 7
- 3.5 Correctness Validation . . . 7
- 3.6 Measurement . . . 7
- 3.7 Roofline Model . . . 8
- 3.8 V3 Tuning . . . 8
- 3.9 Controlled Conditions and Deviations . . . 8

4 Results **9**
- 4.1 Correctness . . . 9
- 4.2 Set A Results . . . 10
- 4.3 Set B Results . . . 12
- 4.4 Roofline Analysis . . . 15
- 4.5 CPU–GPU Crossover . . . 16
- 4.6 OpenMP Scaling . . . 18

5 Discussion **19**
- 5.1 Matrix-Based Methods . . . 19
- 5.2 Direct Convolution . . . 19
- 5.3 Recommendations . . . 20

6 Limitations **20**

7 Conclusion **21**

References **21**

1. Introduction

1.1 Background

Convolutional neural networks (CNNs) are the standard model for image recognition, and most of the time they take to run is spent in their convolution layers. A convolution slides a small filter, usually 3×3, over every position of an input image and multiplies and adds the overlapping values. Written as simple nested loops this is slow, so CNNs are normally run on GPUs. A GPU is only fast, however, when the work and the data are arranged to suit its hardware, and in particular its small on-chip shared memory and its need for neighbouring threads to read neighbouring memory addresses.

Several ways of arranging convolution for a GPU have been proposed. The most common turns the convolution into one large matrix multiplication (im2col) [1], which GPUs perform very efficiently but which copies the input many times. NVIDIA's cuDNN library builds that matrix in small pieces inside shared memory to avoid the memory cost [2]. Other work reduces the extra memory [3], returns to direct convolution with no extra memory at all [4], or uses mathematical transforms such as FFT [5] and Winograd [6] to reduce the number of multiplications.

1.2 Problem Statement

Each of these methods has been studied on its own, usually on different hardware and against different baselines, and the fastest library, cuDNN, is closed source. There was therefore no clear, evidence-based answer to a practical question: for a given convolution layer on a given GPU, which method should be used, and what does it cost in memory?

1.3 Research Gaps

Our literature review identified five gaps:

Gap 1. Memory-efficient direct convolution [4] was designed for CPUs and had not been adapted to the GPU's memory system and compared openly with matrix-based methods.

Gap 2. There was no fair, open, side-by-side comparison of im2col, implicit GEMM and direct convolution on the same GPU.

Gap 3. Speed and memory use were rarely measured together as layer dimensions changed.

Gap 4. Results reported which method was faster, but not why. The reviewed papers did not use the roofline model [7] to show whether a method was limited by memory bandwidth or by computing power.

Gap 5. The cost of moving data between the CPU and the GPU was ignored, so it was unknown at what input size the GPU starts to outperform a multi-threaded CPU.

1.4 Research Question and Objectives

The project addressed this research question: *for which kinds of convolution layers is each method the best choice on a GPU, when both speed and memory are taken into account, and why?*

It had four objectives. O1: implement im2col with matrix multiplication, implicit GEMM and direct convolution in CUDA with the same level of optimisation (Gaps 1 and 2). O2: measure execution time and memory together across layer sizes, including layers from real networks (Gap 3). O3: use the roofline model to explain whether each method is memory-bound or compute-bound (Gap 4). O4: measure end-to-end time, including data transfer, against a multi-threaded CPU version (Gap 5).

1.5 Scope

The study covered the forward pass of a single convolution layer in 32-bit floating point (FP32) on one GPU. FFT and Winograd were not implemented: FFT mainly pays off for large filters, and Winograd is specialised for small filters with a stride of one, so neither is a general-purpose method. Training, lower-precision formats, tensor cores and multi-GPU systems were outside the scope.

2. Literature Review

2.1 Convolution as Matrix Multiplication

Chellapilla et al. [1] showed that convolution can be rewritten as a matrix multiplication. Each input patch is unrolled into one column of a large matrix, and the filters form a second matrix, so that their product gives the output. This technique, im2col, lets convolution use highly tuned GEMM routines from linear algebra libraries. Its weakness is memory: neighbouring patches overlap, so each input value is copied into the matrix up to K^2 times for a $K \times K$ filter, and the matrix can be many times larger than the input.

2.2 Implicit GEMM

Chetlur et al. [2] described cuDNN and its implicit GEMM approach. The im2col matrix is never stored in full. Instead, the small tile of it needed at each moment is formed on the fly in on-chip shared memory, which keeps the speed of tiled GEMM and removes most of the memory cost. cuDNN is the production reference for this work, but it is closed source, and it chooses among several algorithms internally, so its results can be measured but not fully explained.

2.3 Memory-Efficient Convolution

Cho and Brand [3] proposed MEC, which replaces the im2col matrix with a compact representation and evaluates it lazily. It showed that much of the im2col memory is redundant, and that reducing it can also improve speed. Zhang et al. [4] went further and returned to direct convolution with no extra memory at all. By reordering the loops and writing dedicated microkernels, they matched or exceeded GEMM-based convolution on CPUs. Their design relies on the CPU's cache hierarchy and vector registers, however, and it has not been adapted to the GPU, where the corresponding resources are shared memory and thread blocks.

2.4 Transform Methods

Mathieu et al. [5] used FFTs, which turn convolution into element-wise multiplication in the frequency domain and pay off mainly for large filters. Lavin and Gray [6] applied Winograd's minimal filtering algorithms, which reduce the number of multiplications for small filters and are specialised to a stride of one. Modern networks such as VGG [8] and ResNet [9] mostly use small filters, but also 1×1 filters and non-unit strides, so neither transform covers every layer, and this study did not implement them.

2.5 Performance Modelling

Williams et al. [7] introduced the roofline model, which plots a kernel's throughput against its arithmetic intensity, the number of floating-point operations per byte moved to or from memory. Two limits are drawn: peak memory bandwidth, which caps kernels of low intensity, and peak computing rate, which caps kernels of high intensity. The model shows why a kernel performs as it does, and how much room remains. None of the convolution papers above used it, which is the basis of Gap 4.

2.6 Summary

Together, these works show a clear trade-off between the speed of tuned GEMM, the memory cost of im2col, and the appeal of methods that avoid that cost. They do not, however, compare these methods fairly on one GPU, measure speed and memory together, explain the results, or count data transfer. This project addressed those gaps.

3. Methodology

3.1 Research Design

This was a quantitative, experimental and comparative study. Each method was implemented, checked for correctness, and run under controlled conditions while one layer dimension at a time was changed. The independent variables were the method and the layer dimensions. The dependent variables were execution time, memory use, achieved bandwidth and throughput. The hardware, number format, compiler settings and measurement procedure were controlled.

3.2 Implementations

Seven versions were compared (Table 1). All used FP32 and the same NCHW layout, and supported stride and padding. V1 and V2 shared one hand-written GEMM kernel, which tiles the output into 64×64 blocks with shared-memory loading and 4×4 register tiling in each thread, so any difference between them comes from the method rather than from the quality of the code. V3 computes a 16×16 tile of output pixels for a group of CO_B output channels at a time, from an input tile and weights held in shared memory.

Version	Runs on	Description
V1. im2col + GEMM	GPU	Copies input patches into a matrix, then multiplies it with our tiled GEMM
V2. Implicit GEMM	GPU	The same GEMM, with matrix elements computed as they are loaded; the matrix is never stored
V3. Direct convolution	GPU	Tiled kernel over output pixels and channels; no extra matrix
V4. im2col + cuBLAS	GPU	As V1, with NVIDIA's cuBLAS for the multiplication
V5. cuDNN via PyTorch	GPU	`F.conv2d`, with cuDNN choosing its own algorithm
V6. Plain C++	CPU	Nested loops on one core, with no SIMD intrinsics
V7. OpenMP	CPU	V6 parallelised over batch and output channels

Table 1. Versions compared in this study

3.3 Experimental Setup

All reported results were collected on one laptop (Table 2). Code was compiled with `nvcc -O3` for compute capability 8.9. Group members used Google Colab for development only, and no Colab timing appears in this report.

Component	Specification
Machine	HP Victus 15 laptop, Windows 11 Home (build 26200)
CPU	Intel Core i5-13420H, 4 performance and 4 efficiency cores, 12 threads
Memory	15.6 GB RAM
GPU	NVIDIA GeForce RTX 4050 Laptop GPU, 6141 MiB, 20 SMs, driver 592.82
GPU power limit	42.15 W to 50 W across sessions, as reported by `nvidia-smi`
GPU software	CUDA Toolkit 13.4 (nvcc 13.4.59), cuBLAS
CPU compiler	MSVC 19.51 (Visual Studio 2026), OpenMP 2.0
Reference framework	Python 3.11, PyTorch 2.5.1 (CUDA 12.1), cuDNN 9.1.0

Table 2. Hardware and software

3.4 Workloads

Set A was a synthetic sweep. From a baseline layer A00 (56×56 input, 64 input and output channels, 3×3 filter, batch 8), one dimension was changed at a time: input size 14, 28 and 56; channels 16, 64, 128 and 256; filter size 1, 3, 5 and 7; and batch size 1, 8 and 32. That gave 11 layers. Set B held 12 layers from VGG-16 [8] and ResNet-18 [9] at batch sizes 1 and 8 (Table 3). Six further small configurations with sizes that are not multiples of the tile sizes were used only to catch boundary errors. A05 and B04 are the same layer, so both were run.

Layer	Batch 1	Batch 8	Input	Channels	Filter	Stride
VGG-16 conv1_2	B01	B02	224×224	64 → 64	3×3	1
VGG-16 conv3_2	B03	B04	56×56	256 → 256	3×3	1
VGG-16 conv5_2	B05	B06	14×14	512 → 512	3×3	1
ResNet-18 conv1	B07	B08	224×224	3 → 64	7×7	2
ResNet-18 conv3_x	B09	B10	28×28	128 → 128	3×3	1
ResNet-18 conv4_x downsample	B11	B12	28×28	128 → 256	1×1	2

Table 3. Real network layers (Set B)

3.5 Correctness Validation

Inputs and weights were random values between −1 and 1 from a fixed seed. The reference output was computed by PyTorch in double precision on the CPU and stored as FP32. An output passed if every element satisfied $|\text{out} - \text{ref}| \leq 0.001 + 0.001 \times |\text{ref}|$. Every version was checked on all 29 configurations before timing, and benchmark mode repeated the comparison after timing.

3.6 Measurement

Kernel time was measured with CUDA events (V1 to V4) and `torch.cuda.Event` (V5) around the convolution, and with a wall-clock timer for the CPU versions. End-to-end time covered copying the input and weights to the GPU, the convolution, and copying the output back, using a wall-clock timer. It was measured with ordinary pageable host memory and with pinned memory. Each measurement used five warm-up runs and ten timed runs, and we report the median. Peak memory for V1 to V4 was the sum of all allocations, cross-checked with `cudaMemGetInfo`. For V5 it was `torch.cuda.max_memory_allocated()`, which includes cuDNN's workspace. TF32 was turned off and cuDNN's benchmark mode turned on. V7 was run with 1, 4, 8 and 12 threads, and its best time is used for comparison.

Throughput counted $2 \times N \times C_{out} \times H_{out} \times W_{out} \times C_{in} \times K^2$ operations for every version, so a method that performs fewer operations internally shows a higher figure than the operations it actually executes.

3.7 Roofline Model

Peak bandwidth and peak compute were measured on the GPU, not taken from the specification: a float4 copy kernel gave 175.5 GB/s, against a theoretical 192 GB/s, and a fused multiply-add kernel with eight independent accumulators gave 10,799 GFLOP/s. The ridge point was therefore 61.5 FLOP/byte. Bytes moved were estimated from the layer dimensions: the input and weights read once and the output written once, plus one write and one read of the im2col matrix for V1 and V4. These are minimum-traffic estimates. Nsight Compute, an optional cross-check, was not available in the installed toolkit, so the estimates were not compared with hardware counters.

3.8 V3 Tuning

CO_B values of 4, 8 and 16 were compared on A00 and B04 (Table 4). CO_B = 16 gave the lowest total, 7.08% below CO_B = 8. The tuning rule kept CO_B = 8 unless the best value was more than 2% better, so CO_B = 16 was used for all results.

CO_B	A00 (ms)	B04 (ms)	Sum (ms)
4	2.46	39.0	41.5
8	1.94	30.1	32.0
16	1.81	28.0	29.8

Table 4. V3 kernel time for each CO_B value

3.9 Controlled Conditions and Deviations

Runs were made on mains power in the laptop's performance mode, with Windows set to best performance and other applications closed. A short warm-up of a few milliseconds was not enough to bring the GPU out of its idle clocks: the first layer of each run was slower, and peak compute measured only 7,779 GFLOP/s, so we added a spin-up that loads both the GPU's compute units and its memory, for 3 s at the start of a run and 0.5 s before each timed row.

Four other departures from the original plan are recorded here. First, Set B CPU runs used one warm-up run rather than five, because they were long; the ten timed runs were unchanged. Second, V7 with one thread was run on Set B only for B04, the only Set B layer that needs it. Third, GPU cooldowns between commands lasted at least 60 s and until the GPU reached 48 °C or three minutes had passed, rather than a fixed five minutes. Fourth, the Set A CPU run was paused once during layer A08, so the maximum time in the V6 row for A08 is not a real measurement. Its median, which is the value used, is unaffected.

4. Results

4.1 Correctness

All seven versions passed all 29 configurations (Table 5), and V3 also passed with each CO_B value, giving 377 of 377 rows passed. The largest error was 3.13×10^{-4}, well inside the tolerance, and all 115 benchmark rows for V1 to V5 passed their check after timing. No configuration ran out of memory.

Version	Largest max_abs_err	Configurations passed
V1	3.13×10^{-4}	29 / 29
V2	3.13×10^{-4}	29 / 29
V3	3.13×10^{-4}	29 / 29
V4	3.13×10^{-4}	29 / 29
V5	1.07×10^{-4}	29 / 29
V6	2.82×10^{-4}	29 / 29
V7	2.82×10^{-4}	29 / 29

Table 5. Correctness over all 29 configurations

A second, complete pass of the GPU benchmarks was run only to measure repeatability (Table 6). Our own versions, V1 to V3, repeated closely. V4 and V5 were less stable, mainly on very small layers: V5 on B11 took 0.0768 ms in the final pass and 0.0237 ms in the repeat. V4 on B04 took 8.33 ms and then 6.35 ms. Figures for those two versions on small layers should therefore be read with care.

Version	Median diff. (%)	Max diff. (%)	Layer of max	Rows over 10%
V1	0.490	9.25	B01	0 / 23
V2	0.552	4.98	A08	0 / 23
V3	0.908	6.99	A09	0 / 23
V4	0.901	52.0	B11	5 / 23
V5	4.45	224	B11	8 / 23

Table 6. Kernel-time difference between two full passes

We ran the planned sanity checks on the final results. Five passed outright, and four flagged rows (Table 7).

Check	Rows	Finding
Below measured peaks	2	A06 reached 175.7 GB/s (V4) and 180.8 GB/s (V5) against a peak of 175.5 GB/s. This 12 MB layer may be partly served from cache, which the byte estimate ignores; this was not confirmed
`cudaMemGetInfo` within 10% or 2 MB	20	All read a few MB high, which is consistent with allocation rounding across up to four buffers
V1 to V4 not over 2× faster than V5	3	All on B11, where V5 was unstable (above)
Maximum within 1.5× of median	14	All were kernels of under 1 ms, where timer and scheduling jitter is large; medians are unaffected

Table 7. Sanity checks that flagged rows

4.2 Set A Results

Table 8 and Figures 1 and 2 show the synthetic sweep. V5 (cuDNN) was fastest on 10 of 11 layers, and V4 (im2col + cuBLAS) was fastest on A01. Among V1 (im2col + GEMM), V2 (implicit GEMM) and V3 (direct convolution), V2 was fastest on every layer except A03, the 16-channel layer, where V3 took 0.134 ms against 0.242 ms for V2.

The dimensions affected the methods differently. Moving from A03 to A05 multiplies the operations by 256, because channels grow on both sides of the filter. V3 slowed by 209 times, V2 by 62 times and V4 by 34 times, so direct convolution suffered most from many channels. Memory followed the design of each method: V1 and V4 grew with K^2, reaching 313 MB on A08 against 13.0 MB for V2 and V3, a ratio of 24. V5 stayed close to V2 and V3 at large sizes, but needed more on small layers.

		Kernel time (ms)					Peak memory (MB)				
Layer	GFLOP	V1	V2	V3	V4	V5	V1	V2	V3	V4	V5
A00	1.85	1.47	0.939	1.81	1.17	0.225	67.5	12.4	12.4	67.5	20.9
A01	0.116	0.134	0.121	0.178	0.100	0.107	4.35	0.906	0.906	4.35	9.42
A02	0.462	0.419	0.301	0.518	0.318	0.0896	17.0	3.20	3.20	17.0	12.5
A03	0.116	0.372	0.242	0.134	0.231	0.0660	16.9	3.07	3.07	16.9	12.0
A04	7.40	4.81	3.80	7.06	2.21	0.741	135	25.1	25.1	135	34.9
A05	29.6	16.4	15.1	28.0	7.86	2.93	272	51.2	51.2	272	65.6
A06	0.206	0.175	0.119	0.418	0.146	0.0711	18.4	12.3	12.3	18.4	20.4
A07	5.14	3.97	2.58	4.80	3.44	0.661	166	12.6	12.6	166	46.8
A08	10.1	8.50	5.11	9.36	7.24	1.29	313	13.0	13.0	313	47.1
A09	0.231	0.216	0.166	0.330	0.140	0.0517	8.56	1.67	1.67	8.56	10.2
A10	7.40	6.33	3.87	7.03	4.71	0.965	270	49.1	49.1	270	57.7

Table 8. Set A kernel time and peak memory

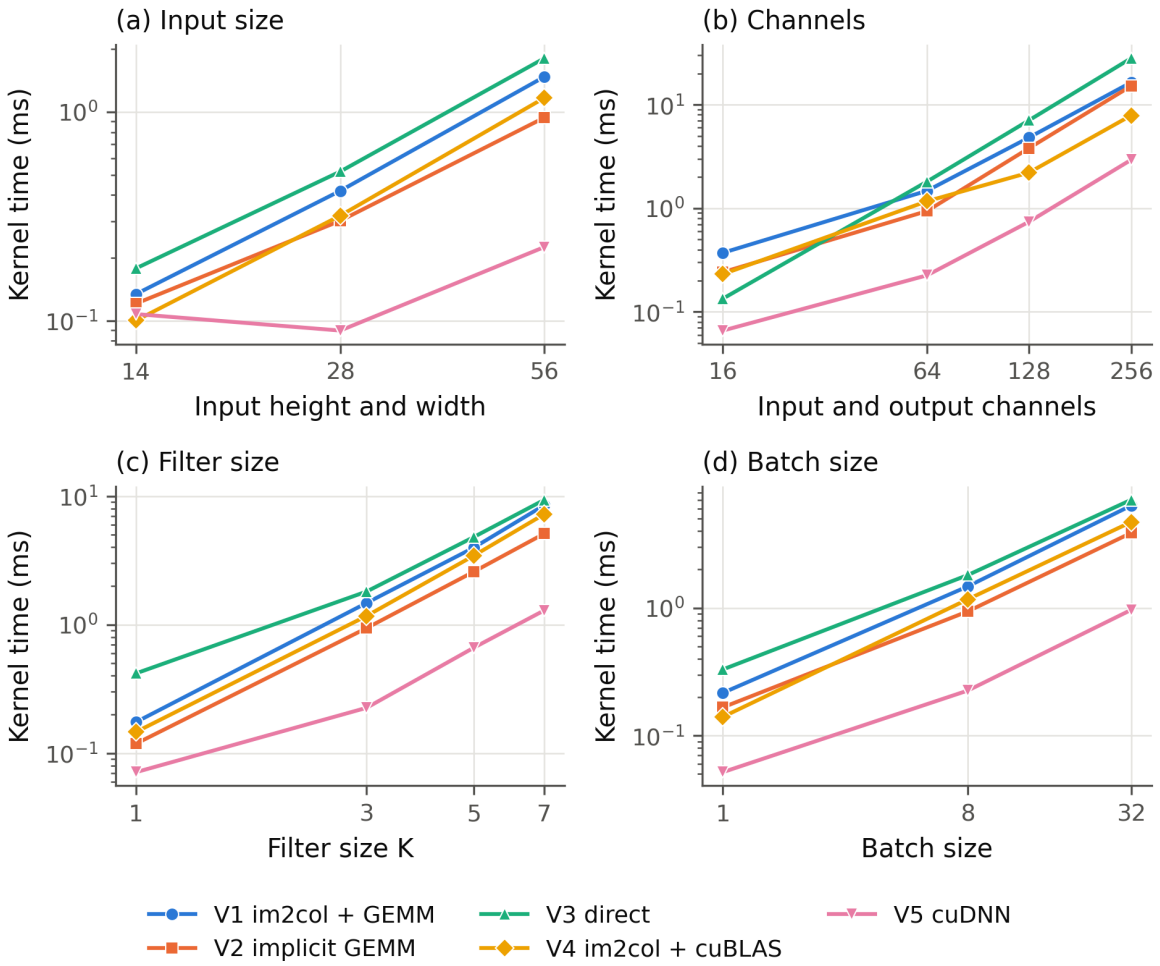


Figure 1. Kernel time across the Set A sweeps

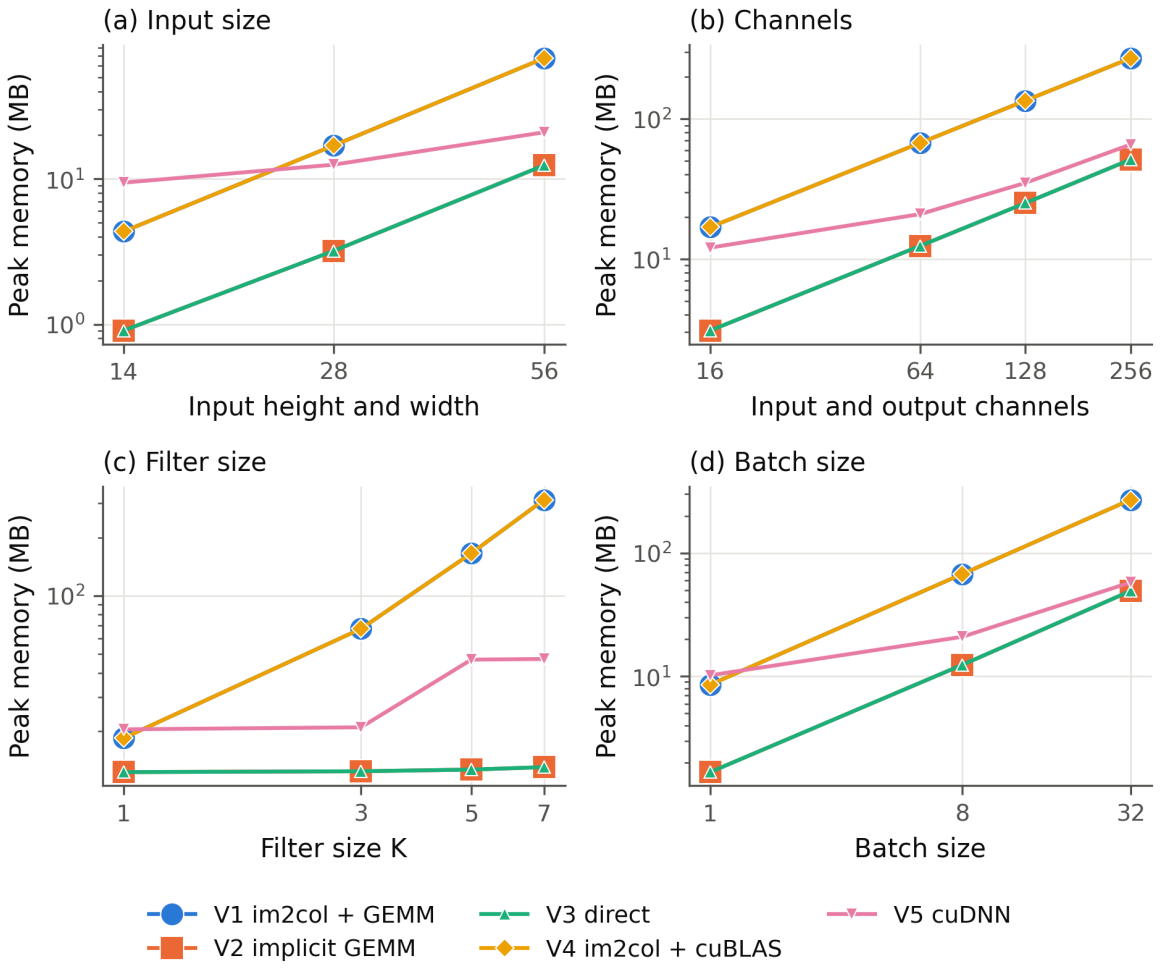


Figure 2. Peak memory across the Set A sweeps. V1 and V4 are equal, as are V2 and V3

4.3 Set B Results

Table 9 and Figure 3 show the real layers. V5 was fastest on 10 of 12 layers and V4 on B05 and B11. V2 was the fastest of V1 to V3 on eight layers, and V1 on B05, B06, B09 and B11. Taking the median over Set B, V4 ran at 0.481 of the speed of V5, V2 at 0.238, V1 at 0.220 and V3 at 0.145 (Table 11).

Peak memory (Table 10 and Figure 4) separated the methods clearly. On B02, V1 and V4 needed 1,080 MB, V2 and V3 needed 196 MB, and V5 needed 205 MB. On small layers cuDNN's workspace outweighed the data: on B09, V5 needed 11.0 MB against 1.33 MB for V2 and V3.

Speedups over V6 ranged from 113 to 9,300 times. V6 is a deliberately plain, non-SIMD baseline, however, so these figures flatter the GPU. The fairer CPU comparison is V7 (Section 4.5).

Layer	V1	V2	V3	V4	V5	V6	Best V7
B01	2.83	1.85	2.75	2.00	0.380	3,340	771
B02	24.6	15.6	21.5	21.2	3.90	26,400	5,960
B03	1.95	1.92	3.70	0.842	0.386	3,450	746
B04	16.4	15.2	28.0	8.33	2.93	27,300	5,490
B05	0.728	0.934	1.40	0.231	0.311	737	165
B06	4.56	4.85	7.49	1.85	0.934	5,880	1,270
B07	0.207	0.137	0.181	0.130	0.0681	175	37.2
B08	1.57	1.01	1.35	1.25	0.370	1,390	279
B09	0.220	0.240	0.356	0.101	0.0543	187	39.4
B10	1.23	1.05	1.85	0.638	0.227	1,510	307
B11	0.0193	0.0235	0.156	0.0154	0.0768	17.6	3.30
B12	0.0881	0.0838	0.412	0.0541	0.0435	142	26.6

Table 9. Set B kernel time (ms). Best V7 is the lowest time over all thread counts

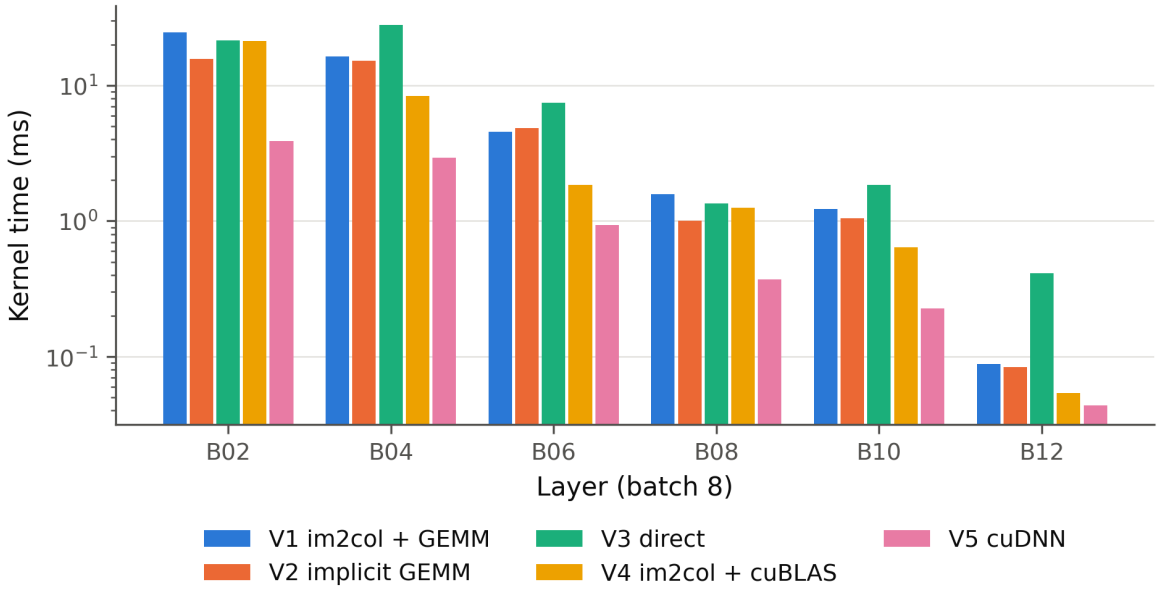


Figure 3. Kernel time on the batch-8 layers of Set B

Layer	V1	V2	V3	V4	V5
B01	135	24.6	24.6	135	33.2
B02	1,080	196	196	1,080	205
B03	35.9	8.38	8.38	35.9	23.6
B04	272	51.2	51.2	272	65.6
B05	13.2	9.77	9.77	13.2	17.9
B06	42.7	15.1	15.1	42.7	50.1
B07	10.7	3.67	3.67	10.7	12.7
B08	85.4	29.1	29.1	85.4	37.3
B09	4.77	1.33	1.33	4.77	11.0
B10	34.2	6.69	6.69	34.2	17.2
B11	0.795	0.699	0.699	0.795	8.82
B12	5.48	4.72	4.72	5.48	13.7

Table 10. Set B peak memory (MB)

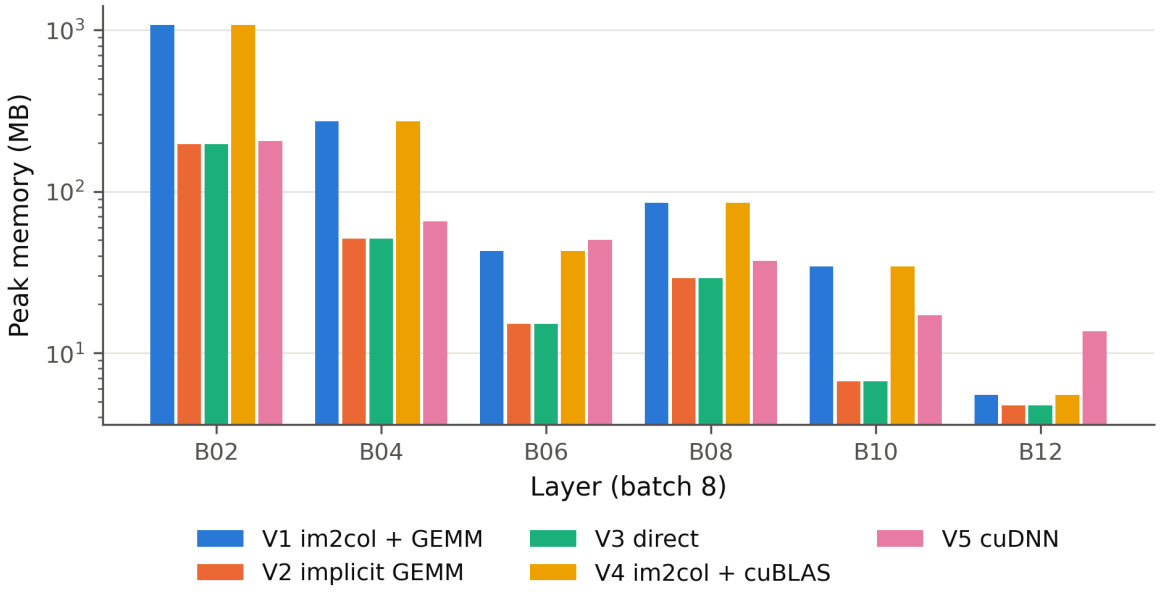


Figure 4. Peak memory on the batch-8 layers of Set B

	Speedup over V6					Speedup over V5			
Layer	**V1**	**V2**	**V3**	**V4**	**V5**	**V1**	**V2**	**V3**	**V4**
B01	1,180	1,810	1,220	1,670	8,800	0.134	0.206	0.138	0.190
B02	1,070	1,690	1,230	1,250	6,770	0.159	0.250	0.181	0.184
B03	1,770	1,800	933	4,100	8,960	0.198	0.200	0.104	0.458
B04	1,660	1,800	975	3,280	9,300	0.179	0.194	0.105	0.352
B05	1,010	789	525	3,180	2,370	0.427	0.333	0.221	1.34
B06	1,290	1,210	785	3,170	6,300	0.205	0.193	0.125	0.504
B07	843	1,270	964	1,340	2,570	0.328	0.496	0.376	0.524
B08	882	1,370	1,030	1,110	3,750	0.235	0.366	0.274	0.295
B09	852	781	525	1,860	3,450	0.247	0.227	0.152	0.539
B10	1,230	1,440	818	2,370	6,640	0.185	0.217	0.123	0.356
B11	914	749	113	1,140	229	3.98	3.26	0.493	4.99
B12	1,610	1,690	344	2,620	3,260	0.494	0.519	0.106	0.804

Table 11. Speedup on Set B. Below 1 means slower than V5. V5 on B11 was unstable

4.4 Roofline Analysis

Figure 5 and Table 12 place the methods on the roofline. With the ridge point at 61.5 FLOP/byte, V1 and V4 were memory-bound on 22 of 23 layers, because the extra im2col traffic gives them a median arithmetic intensity of 14.4 FLOP/byte. V2, V3 and V5 move far fewer bytes, with a median intensity of 143 FLOP/byte, and were compute-bound on 18 of 23 layers. Their five exceptions were the layers with 1×1 filters or few channels: A03, A06, B07, B11 and B12. B07 lay just under the ridge, at 61.3 FLOP/byte. The one exception for V1 and V4 was B06.

The position relative to the roof matters as much as the classification. V5 reached a median of 70.2% of peak compute, and V4 a median of 57.7% of peak bandwidth. V2 and V3 reached only 16.4% and 9.27% of peak compute, and just 6.24% and 3.04% of peak bandwidth. They were limited by neither roof.

Version	**Memory-bound**	**Compute-bound**	**Median AI**	**GFLOP/s**	**% peak**	**GB/s**	**% peak**
V1	22	1	14.4	1,200	11.1	75.0	42.7
V2	5	18	143	1,770	16.4	11.0	6.24
V3	5	18	143	1,000	9.27	5.33	3.04
V4	22	1	14.4	1,650	15.3	101	57.7
V5	5	18	143	7,580	70.2	33.9	19.3

Table 12. Roofline summary over all 23 layers. AI is in FLOP/byte; values are medians

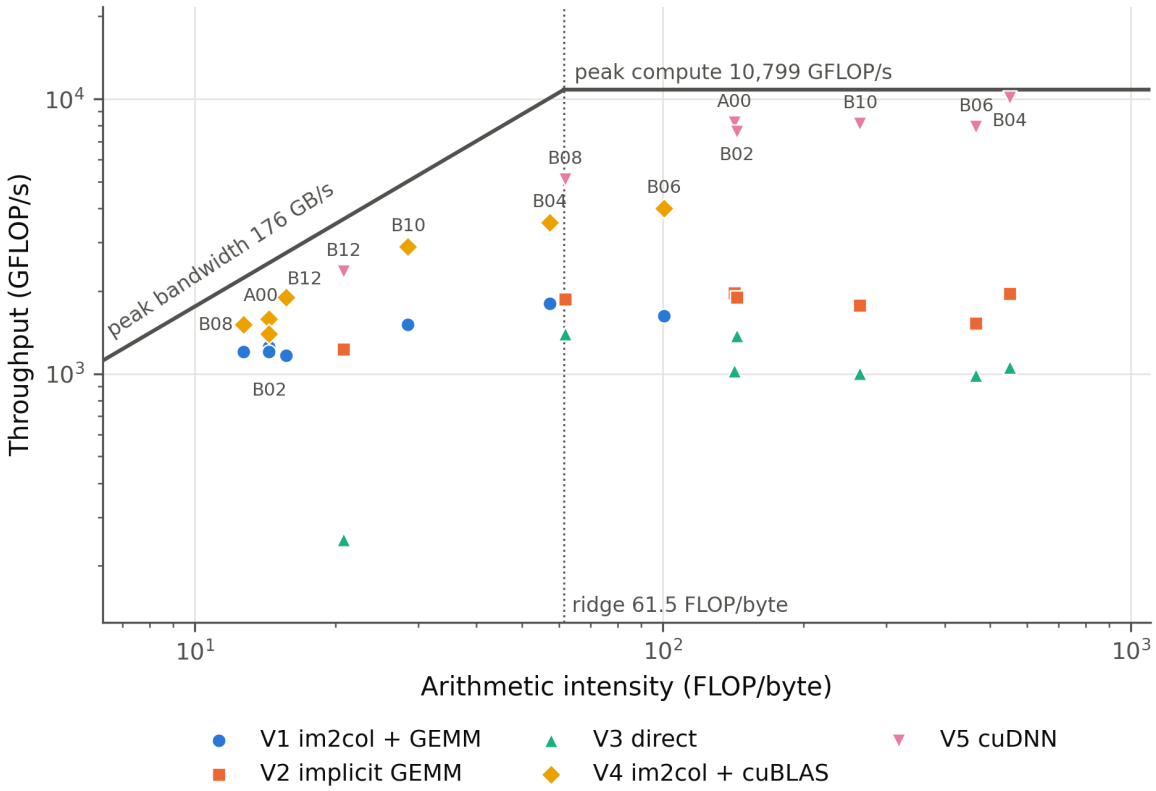


Figure 5. Roofline for A00 and the batch-8 layers of Set B

4.5 CPU–GPU Crossover

Every GPU version beat the best OpenMP time end to end on every Set A layer, with both pageable and pinned memory (Table 13 and Figure 6). The crossover was therefore below the smallest layers tested, 0.116 GFLOP, and could not be located within Set A. Even there, the fastest of V1 to V3 was at least 29.7 times faster than V7 end to end.

Data transfer was still a large cost (Table 14). Pinned memory lowered end-to-end time on every row, with a median ratio of pageable to pinned time of 1.83 for V5 and between 1.22 and 1.42 for V1 to V4. On B02, transfer dominated: V5's kernel took 3.90 ms, but its pageable end-to-end time was 47.4 ms, slightly more than the 46.6 ms of V2, whose kernel was four times slower. With pinned memory V5 regained the lead, at 24.2 ms against 30.4 ms.

Layer	GFLOP	Best V7	Fastest	Pageable	Pinned	V5 pageable
A01	0.116	19.3	V2	0.359	0.298	0.309
A03	0.116	18.5	V3	0.624	0.482	0.760
A06	0.206	68.7	V2	1.90	1.16	2.84
A09	0.231	45.0	V2	0.550	0.348	0.480
A02	0.462	78.4	V2	0.869	0.624	0.793
A00	1.85	433	V2	2.95	2.04	3.08
A07	5.14	926	V2	4.65	3.78	2.80
A04	7.40	1,480	V2	8.03	5.88	5.43
A10	7.40	1,420	V2	12.1	7.84	11.6
A08	10.1	1,600	V2	7.31	6.25	3.58
A05	29.6	6,230	V2	23.0	18.8	14.0

Table 13. End-to-end time on Set A in ms, sorted by size. Fastest is the fastest of V1 to V3 by kernel time, and Pageable and Pinned are its times

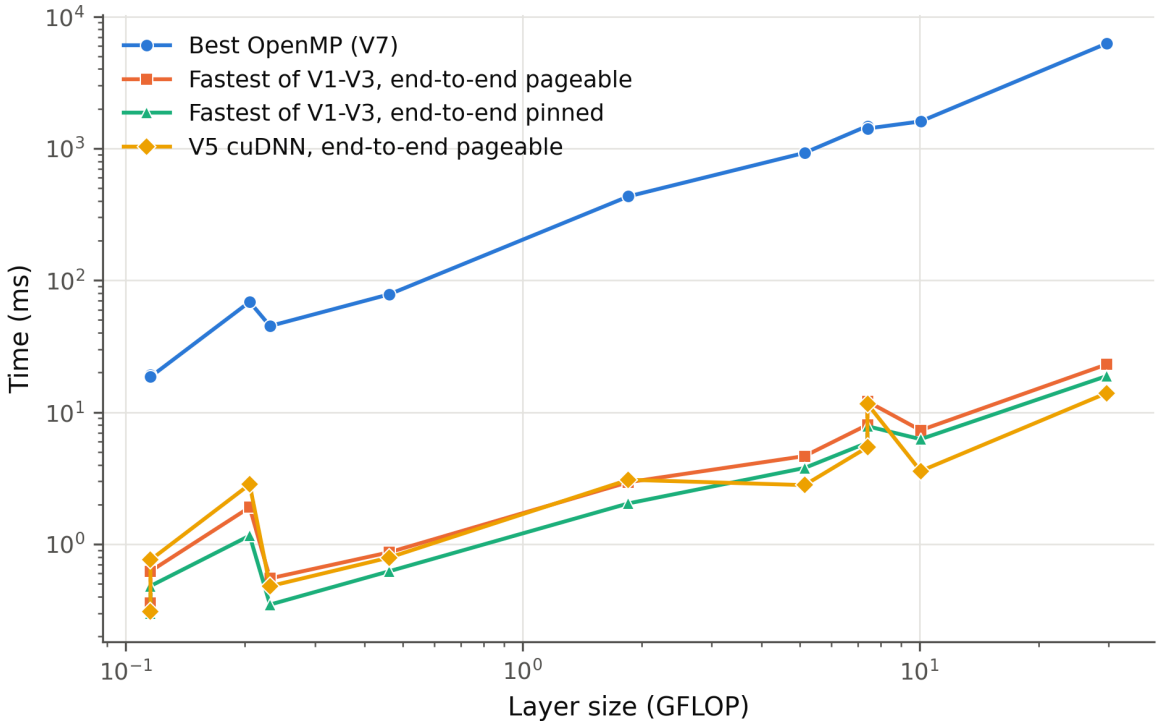


Figure 6. End-to-end time against layer size on Set A

Layer	Version	Kernel (ms)	Pageable (ms)	Pinned (ms)
B02	V1	24.6	53.8	41.4
B02	V2	15.6	46.6	30.4
B02	V3	21.5	54.1	37.7
B02	V4	21.2	49.9	33.1
B02	V5	3.90	47.4	24.2
B04	V1	16.4	23.1	19.5
B04	V2	15.2	23.3	18.8
B04	V3	28.0	36.7	32.4
B04	V4	8.33	14.9	11.0
B04	V5	2.93	13.9	7.58

Table 14. Kernel and end-to-end time on the two largest layers

4.6 OpenMP Scaling

Scaling was well below ideal (Table 15 and Figure 7). With 12 threads, V7 ran 3.73 times faster than with one thread on A00, and 4.90 times faster on B04. Between 8 and 12 threads, A00 barely improved. Best V7 was 3.55 to 5.34 times faster than V6 over all layers. The i5-13420H has 4 performance and 4 efficiency cores, and only the performance cores have a second hardware thread, so the 12 threads are not equal.

Layer	1 thread	4 threads	8 threads	12 threads
A00	1,610	553	437	433
B04	26,900	11,000	7,320	5,490

Table 15. V7 kernel time (ms) by thread count

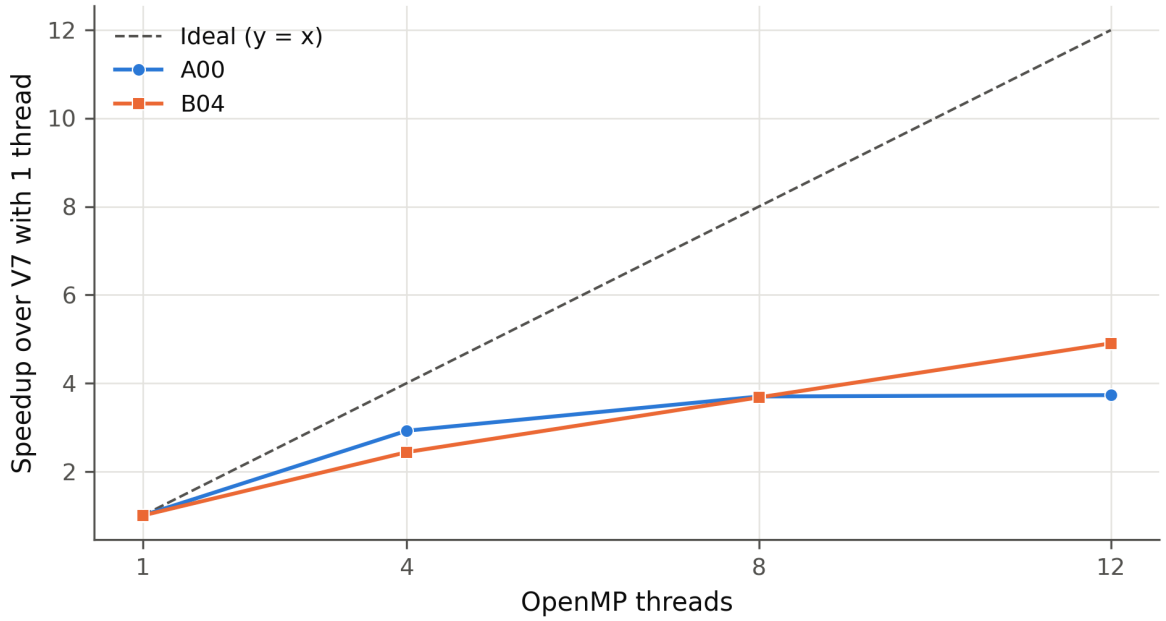


Figure 7. OpenMP scaling on A00 and B04

5. Discussion

5.1 Matrix-Based Methods

V1 (im2col + GEMM) and V2 (implicit GEMM) share the same GEMM, so their difference isolates the method. V2 skips the im2col pass, and the benefit grew with the size of the matrix relative to the multiplication: 15.6 ms against 24.6 ms on B02, where the matrix has about 231 million elements, but only 15.2 ms against 16.4 ms on B04, where the multiplication dominates. On B05, B06, B09 and B11, V1 was faster than V2. These layers have small spatial sizes and many channels. A likely cause is that V2 computes an index for every element it loads, which costs more than V1's direct read when the matrix is small enough to read cheaply.

V4 (im2col + cuBLAS) uses V1's im2col, so its gain over V1 shows what expert tuning adds. That gain was large on many-channel layers, such as 8.33 ms against 16.4 ms on B04, and small on B02, at 21.2 ms against 24.6 ms. This suggests that our GEMM is competitive when the matrix is wide, and that cuBLAS gains most when the number of output channels is large. On the roofline, V1 and V4 are memory-bound for one reason: the im2col matrix is written and then read. That is a property of the method, so tuning the multiplication cannot remove it, and V4 reached a median of only 57.7% of peak bandwidth.

V5 (cuDNN) ran at about four times the speed of V2 in the median. Our V2 and V5 use similar memory on large layers, so V5 is not simply trading memory for speed. It may be using Winograd or other methods internally [6], in which case it performs fewer operations than the count we use and its 70.2% of peak compute overstates its real rate. Because cuDNN is closed source, we cannot confirm this.

5.2 Direct Convolution

V3 addressed Gap 1. It used the same memory as V2 on every layer, with no workspace, and it was correct on all configurations, so the zero-overhead property of direct convolution [4] carried over to the GPU. It was usually slower, however, at a median of 0.145 of the speed of V5.

The results show where it works and where it does not. V3 was fastest of V1 to V3 on A03, and on layers with large images and few channels it was close to or better than V1: 21.5 ms against 24.6 ms on B02, and 1.35 ms against 1.57 ms on B08. It was weakest with many channels, at 28.0 ms against 15.2 ms for V2 on B04, because the kernel reloads the input tile and weights and passes two barriers for every input channel, in every group of output channels. A GEMM instead moves 16 inner elements at once. V3 was also weak on the 1×1, stride-2 layers: on B12 it took 0.412 ms against 0.0838 ms for V2. There, each input tile covers 31×31 positions, of which only a 16×16 pattern is used, so most of the loaded data is wasted.

V3 reached only 3.04% of peak bandwidth and 9.27% of peak compute in the median, so its limit is not memory or arithmetic capacity. It is the overhead of loading and synchronising, which means the method has room to improve. Tiling the input channels as well, for instance, would reduce the number of loads and barriers. This remains future work.

5.3 Recommendations

Table 16 applies the agreed rule to Set B: recommend the fastest of V1 to V3, unless it uses more than twice the memory of the second-fastest while being less than 10% faster, in which case recommend the second-fastest. V2 and V3 had the lowest memory on every layer. V5 and V4 are excluded because this table compares the three open methods.

The rule changed the result on B06 and B09, where V1 was slightly faster but V2 used far less memory. V2 was recommended for 10 of 12 layers. The second pass gave the same recommendation on 11 layers. It differed on B12, where V1 and V2 were within 4.83% of each other.

Layer	Fastest	Lowest memory	Second	Lead (%)	Recommended
B01	V2	V2/V3	V3	32.8	**V2**
B02	V2	V2/V3	V3	27.5	**V2**
B03	V2	V2/V3	V1	1.44	**V2**
B04	V2	V2/V3	V1	7.72	**V2**
B05	V1	V2/V3	V2	22.1	**V1**
B06	V1	V2/V3	V2	5.86	**V2**
B07	V2	V2/V3	V3	24.3	**V2**
B08	V2	V2/V3	V3	25.2	**V2**
B09	V1	V2/V3	V2	8.33	**V2**
B10	V2	V2/V3	V1	14.7	**V2**
B11	V1	V2/V3	V2	18.0	**V1**
B12	V2	V2/V3	V1	4.83	**V2**

Table 16. Recommended method for each Set B layer. Lead is the fastest method's lead over the second-fastest

This answers the research question. When a production library is available, cuDNN is the best choice on nearly every layer, and it should be fed from pinned memory, because on large layers transfer can cancel its kernel advantage. V4 is competitive on small, many-channel layers such as B05 if the im2col memory is acceptable. Among open methods, V2 is the best general choice: it is fastest on most layers and uses the least memory, because it does not store the im2col matrix. V1 is worthwhile only on small inputs with many channels, such as B05, where the matrix is small enough that its memory cost hardly matters. V3 is the choice only where channels are few and code simplicity is valued, since it does not beat V2 on memory.

6. Limitations

All results came from one laptop GPU with a modest 96-bit memory bus. A GPU with more bandwidth would shift the ridge point, and could change which method is best, although the roofline gives a way to reason about this. The reported power limit varied between 42.15 W and 50 W across sessions, which may have affected some results. cuDNN is closed source, so its results can be measured but not fully explained.

The byte counts behind the roofline are minimum-traffic estimates that ignore caching, and

they were not checked against hardware counters. The operation count is the same for every version, which may overstate V5. Peak compute comes from a very short kernel, and A06 slightly exceeded the measured peak bandwidth, as described in Section 4.1.

V4 and V5 were less repeatable than our own versions on small layers, and all times below 1 ms carry noticeable jitter. V3 was tuned on only two layers. The deviations listed in Section 3.9 also apply. The crossover could not be located, because Set A did not extend to small enough layers. The study covered only FP32 forward-pass convolution, and did not include FFT, our own Winograd, lower precision or tensor cores.

7. Conclusion

This project compared seven versions of convolution on one laptop GPU, and all seven passed all 29 correctness configurations.

Gap 1: direct convolution was adapted to the GPU with no workspace, but it was usually slower than V2, limited by its loading and synchronisation overhead. Gap 2: V1, V2 and V3 were compared fairly with shared techniques and open code, and V2 was fastest on 18 of 23 layers. Gap 3: time and memory measured together showed that V2 needs 196 MB where im2col needs 1,080 MB on the largest layer, while V5 needs more than V2 on small layers. Gap 4: the roofline showed that V1 and V4 are memory-bound by the im2col matrix, while V2 and V3 were far below both limits. Gap 5: every GPU version beat OpenMP end to end on all Set A layers, so the crossover lies below 0.116 GFLOP, but transfer can dominate on large layers, and pinned memory reduces it.

The research question therefore has a clear answer. cuDNN is the best choice on most layers where it can be used. V2 is the best open method for most layers, considering both speed and memory, and V1 suits only small inputs with many channels. Future work should reduce the loading overhead in V3, locate the crossover with smaller layers, and repeat the study on a GPU with a wider memory bus.

References

[1] K. Chellapilla, S. Puri, and P. Simard, “High performance convolutional neural networks for document processing,” in *Proc. 10th Int. Workshop on Frontiers in Handwriting Recognition (IWFHR)*, La Baule, France, 2006. [Online]. Available: https://hal.inria.fr/inria-00112631

[2] S. Chetlur, C. Woolley, P. Vandermersch, J. Cohen, J. Tran, B. Catanzaro, and E. Shelhamer, “cuDNN: Efficient primitives for deep learning,” arXiv preprint arXiv:1410.0759, 2014.

[3] M. Cho and D. Brand, “MEC: Memory-efficient convolution for deep neural network,” in *Proc. 34th Int. Conf. on Machine Learning (ICML)*, PMLR vol. 70, pp. 815–824, 2017.

[4] J. Zhang, F. Franchetti, and T. M. Low, “High performance zero-memory overhead direct convolutions,” in *Proc. 35th Int. Conf. on Machine Learning (ICML)*, PMLR vol. 80, pp. 5776–5785, 2018.

[5] M. Mathieu, M. Henaff, and Y. LeCun, “Fast training of convolutional networks through

FFTs," in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2014. arXiv:1312.5851.

[6] A. Lavin and S. Gray, "Fast algorithms for convolutional neural networks," in *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, pp. 4013–4021, 2016.

[7] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.

[8] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2015. arXiv:1409.1556.

[9] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.